

Отчет

Тема и цель

Разработана учебная платформа обработки задач в рамках ЛР1, ЛР2, ЛР3 и ЛР4.

Цель работы: применить ``Protocol``, дескрипторы, ``@property``, итераторы, генераторы и ``asyncio``.

ЛР1: Источники задач и контракт

Контракт источника задач описан через ``typing.Protocol``.

Для runtime-проверки применен декоратор ``@runtime_checkable``.

Реализованы источники ``FileTaskSource``, ``GeneratorTaskSource``, ``APITaskSource``.

Добавлена функция ``process_task_sources(...)`` для агрегации задач.

Реализация расширяема: новые источники добавляются без изменения ядра.

ЛР2: Модель ``Task`` и защита инвариантов

Реализована доменная модель ``Task`` с разделением API и внутреннего состояния.

Валидация полей выполнена через пользовательские data-descriptor.

Проверяются поля ``task_id``, ``description``, ``priority``, ``status``.

Добавлены специализированные исключения валидации.

Базовый класс ошибок: ``TaskValidationError``.

Частные ошибки: ``InvalidTaskIdError``, ``InvalidDescriptionError``,
``InvalidPriorityError``, ``InvalidStatusError``.

Через ``@property`` реализованы ``created_at`` и ``is_ready_to_run``.

Показана разница между data и non-data descriptor (``public_label``).

ЛР3: Очередь задач, итераторы и генераторы

Реализована коллекция ``TaskQueue`` и отдельный итератор ``TaskQueueIterator``.

Обеспечен корректный протокол итерации и обработка ``StopIteration``.

Поддерживается повторный обход: каждый ``__iter__`` возвращает новый итератор.

Реализованы ленивые генераторные методы с ``yield``.

Добавлены фильтры: ``filter_by_status(...)``, ``filter_by_priority(...)``.

Добавлен потоковый метод ``iter_pending()`` для готовых задач.

Генераторный подход снижает расход памяти на больших наборах данных.

ЛР4: Асинхронный исполнитель задач

Реализована асинхронная очередь.

Контракт обработчика ``TaskHandler`` через ``Protocol``.

Реализован ``AsyncTaskExecutor``, управляющий постановкой задач и их выполнением.

Использован ``TaskExecutionContext`` как async context manager.

Добавлено централизованное логирование в файл через ``ExecutionLogger``.

Обрабатываются ошибки выполнения и отсутствия подходящего обработчика.

Архитектура допускает добавление новых обработчиков без изменения ядра.

Тестирование

Для каждой лабораторной подготовлены отдельные тесты ``pytest``.

Файлы тестов: ``test_tasksSources.py``, ``test_task_model.py``,
``test_task_queue.py``, ``test_task_executor.py``.

Проверены позитивные и негативные сценарии.

Покрыты ошибки валидации, поведение очереди, фильтров и обработчиков.

Вывод

Поставленные задачи ЛР1-ЛР4 выполнены в полном объеме.

Получено расширяемое и типизированное решение для обработки задач.
Реализация соответствует требованиям и готова к дальнейшему развитию.
Код структурирован по модулям, покрыт тестами и документирован.